

Où et comment la loi gaussienne intervient dans les modèles (et pourquoi ce n'est pas un output du modèle)

DeepEdgeBenchmark

22 juillet 2026

Introduction

Ce document résume deux questions posées sur le calcul des intervalles de confiance et du CRPS (Continuous Ranked Probability Score) dans les modèles de prévision du projet : où la loi gaussienne est-elle codée, à quoi s'applique-t-elle exactement, quand est-elle appelée, et surtout : est-elle produite directement par le modèle, ou ajoutée après coup par le code ?

1 Où est codée la loi gaussienne, et quand est-elle appelée

Il faut distinguer deux usages complètement séparés dans le code.

1.1 Usage 1 : construire les intervalles de confiance à 95 % (IC95) et les prévisions ponctuelles

C'est l'usage actif, utilisé par tous les modèles pour produire RMSE, IC95 et largeur d'IC95 dans le dashboard. Il n'a pas été modifié par l'ajout du CRPS empirique.

- **ARIMA-GARCH** (`models/arima_model.py`)
 - `arima_model.py`, lignes 152–154 et 164–166 :
`arch_model(resid, vol="Garch", p=1, q=1, dist="normal", rescale=False)`

Hypothèse gaussienne sur les *innovations du GARCH*, utilisée pour estimer par maximum de vraisemblance la volatilité conditionnelle. Appelée au fit initial et à chaque refit (tous les 20 pas par défaut) dans `run_arima_garch`.

- `arima_model.py`, ligne 52 : `Z_95 = 1.96`, appliqué lignes 172–173 :

$$\text{lower} = \text{last_price} \cdot e^{\mu - 1.96 \sigma}, \quad \text{upper} = \text{last_price} \cdot e^{\mu + 1.96 \sigma}$$

Suppose le rendement à 1 pas $\sim \mathcal{N}(\mu, \sigma)$. Appelée à chaque pas du walk-forward (Gate 2, horizon D1) et dans `next_step_arima_garch`.

- `benchmarks/multi_horizon.py`, lignes 68–69 : même formule, réutilisée pour l'horizon D7 et la prévision live (multi-pas, variance GARCH cumulée).
- **LSTM** (`models/lstm_model.py`)
 - Lignes 186–187 :

$$\text{lower} = \text{preds} - 1.96 \sigma_{\text{train}}, \quad \text{upper} = \text{preds} + 1.96 \sigma_{\text{train}}$$

où σ_{train} est l'écart-type des résidus *sur l'ensemble d'entraînement*, calculé **une seule fois**. Appelée à la fin de chaque `run_lstm` (Gate 2, D1) et dans `next_step_lstm`.

- `benchmarks/multi_horizon.py`, ligne 212 : même écart-type, élargi en $\sigma\sqrt{h}$ pour l'horizon D7/live.

- **Naive** (`models/naive_model.py`) : ligne 55, $Z_{95} = 1.96$, appliqué lignes 130–131 : dernier prix $\pm 1.96\sigma$ (marche aléatoire). Hors scope du CRPS (non demandé, baseline triviale).
- **SARIMA** (`models/sarima_model.py`) : aucun 1.96 écrit dans notre code – l’hypothèse gaussienne est **implicite** dans `SARIMAX.get_forecast().conf_int()` (statsmodels), dont le filtre de Kalman suppose des innovations gaussiennes par construction de la classe. Appelée à chaque refit de `run_sarima` (Gate 2, D1) et dans `forecast_from_fitted_sarima` (D7/live).
- **Prophet** (`models/prophet_model.py`) : même chose, pas de 1.96 explicite – `yhat_lower/yhat_upper` proviennent du modèle d’incertitude interne de la librairie Prophet (bruit gaussien sur tendance/saisonnalité par défaut). Appelée à chaque `model.predict()` dans `run_prophet` et en D7/live.
- **TSDiff** (`models/tsdiff_model.py`) : la docstring (ligne 12) parle de prédire “the Gaussian noise” – attention, cela décrit le *bruit de diffusion* que le denoiser apprend à prédire pendant l’entraînement (formulation DDPM/DDIM standard), **pas** la loi de sortie. La distribution prédictive finale de TSDiff est explicitement *non gaussienne* (ligne 19 : “a genuine predictive distribution rather than a Gaussian residual band”) – c’est un nuage de trajectoires simulées, sans hypothèse de forme.

1.2 Usage 2 : un CRPS gaussien fermé, préexistant et séparé

- `honest_eval/metrics.py`, lignes 231–240 : fonction `crps_gaussian(mu, sigma, actual)` – formule fermée du CRPS pour une loi prédictive gaussienne $\mathcal{N}(\mu, \sigma)$ (Gneiting & Raftery, cas gaussien), via `scipy.stats.norm`.
- Utilisée uniquement dans `experiments/pooled_analysis.py` (script d’analyse hors-ligne), qui dérive un σ implicite à partir de la largeur de l’IC95 :

$$\sigma_{\text{implicite}} = \frac{y_{\text{upper}} - y_{\text{lower}}}{2 \times 1.96}$$

puis applique la formule fermée gaussienne.

- **Jamais appelée** dans `model_artifacts/` ni `validation/` – donc jamais dans le pipeline quotidien ni le dashboard. C’est un CRPS *approximé* par hypothèse gaussienne, entièrement distinct du CRPS *empirique* (`crps_empirical`, Gneiting & Raftery éq. 20) calculé par `model_artifacts/crps_kpis.py` à partir des nuages bootstrap/Monte-Carlo-Dropout/TSDiff.

2 La gaussienne est-elle un output du modèle ?

Non. Dans aucun des cinq modèles la loi gaussienne n’est produite directement par le modèle – c’est toujours un choix de code ajouté autour, mais à deux niveaux très différents selon les cas.

2.1 LSTM et Naive : convention post-hoc, sans rapport avec le modèle

Le réseau LSTM se termine par une couche Dense(1) : il ne sort **qu’un seul nombre** (le prix prédit), aucune notion d’incertitude n’est produite par le réseau. La bande $\pm 1.96\sigma$ est construite entièrement à part, après l’entraînement :

```
train_preds = scaler.inverse_transform(model.predict(X_train)...).flatten()
std = np.std(train.values[seq_len:] - train_preds) # UN SEUL nombre
lower = preds - 1.96 * std
upper = preds + 1.96 * std
```

`std` est l'écart-type des erreurs du modèle sur le jeu d'entraînement, calculé **une seule fois**, puis recopié tel quel autour de **chaque** prédiction future – que le modèle soit en réalité plus ou moins confiant à cet instant précis n'entre jamais en jeu. C'est une convention manuelle, pas une information produite par le réseau. Le raisonnement est identique pour Naive.

2.2 ARIMA-GARCH, SARIMA, Prophet : l'hypothèse est dans le modèle statistique, pas dans un réseau de neurones

Ce ne sont pas des réseaux de neurones mais des modèles statistiques classiques, dont la procédure d'ajustement (maximum de vraisemblance) suppose *explicitement* un bruit gaussien pour estimer les paramètres.

- **GARCH** : le mot-clé `dist="normal"` est un choix explicite (on aurait pu écrire `dist="t"` pour une loi de Student, aux queues plus épaisses). Le GARCH estime bien un σ qui varie dans le temps (recalculé à chaque pas via `garch_res.forecast(...)`) – il y a donc un vrai calcul dynamique – mais la forme “gaussienne” de l'intervalle final reste un choix, pas un fait mesuré sur les données.
- **SARIMA** : SARIMAX de statsmodels utilise un filtre de Kalman dont la vraisemblance interne est gaussienne *par construction de la classe* (ce n'est même pas un paramètre réglable dans notre code) – `conf_int()` en ressort directement une borne gaussienne.
- **Prophet** : même principe, modèle bayésien interne à la librairie avec bruit d'observation gaussien par défaut.

Aucun de ces trois modèles n'est un réseau de neurones – ce sont des modèles paramétriques classiques où “gaussien” est une hypothèse de travail choisie au moment de l'ajustement, pas une propriété empirique des rendements observés. Or les rendements financiers sont connus pour avoir des queues plus épaisses qu'une loi normale (*fat tails*) – c'est précisément pour cette raison que le CRPS empirique (par bootstrap des résidus ou Monte Carlo Dropout) est utile : il ne fait aucune de ces hypothèses de forme.

2.3 TSDiff : le seul cas qui échappe complètement à la gaussienne

TSDiff ne sort ni un σ ni une hypothèse de forme : il tire un nuage de trajectoires (`sample_paths`) directement, sans jamais supposer une forme gaussienne pour la distribution prédictive finale. C'est exactement la raison pour laquelle il disposait déjà d'un CRPS empirique avant les autres modèles, et le point de départ de la demande d'étendre ce calcul aux quatre autres modèles.

En résumé

La gaussienne n'est jamais “vue” par le modèle sur les nouvelles données :

- pour **LSTM** et **Naive**, c'est un ruban de largeur fixe, recopié depuis l'erreur d'entraînement, indépendant de chaque nouvelle prédiction ;
- pour **ARIMA-GARCH**, **SARIMA** et **Prophet**, c'est une hypothèse de forme choisie pour pouvoir calculer analytiquement un intervalle à partir d'un σ estimé par le modèle statistique ;
- pour **TSDiff**, il n'y a aucune hypothèse de forme : la distribution prédictive est directement le nuage d'échantillons simulés.